

MASTER DEVELOPMENT BLUEPRINT

PART-1: PROJECT FOUNDATION

School ERP

React + TypeScript + MUI | Java 21 + Spring Boot 3 | PostgreSQL

Prepared by: Solution Architecture / Engineering Leadership Team

Document Version: 1.0

Date: 29 July 2026

Classification: Confidential — Engineering Team Only

1. Introduction

This document is the Master Development Blueprint (MDB) v1.0, Part-1: Project Foundation, for the School ERP System. It is the master implementation guideline that development teams and AI coding assistants follow throughout the build phase, translating the business and architectural decisions already established in RGD v2.0, Appendix-A through Appendix-N, and the eleven approved UI/UX Screen Specifications into a concrete, technology-specific engineering foundation.

Unlike the preceding documentation set — which was deliberately technology-neutral — this MDB is technology-specific: it fixes the stack (Section 3), the architecture's physical shape (Section 4), the repository and package layout (Sections 5-7), and the naming/coding standards (Sections 9-10) that every subsequent line of code must follow. It does not contain source code, API definitions, database table definitions, or entity implementations — those remain the domain of the actual codebase, guided by this blueprint, not this document.

Per the governing rules for this task, this document does not regenerate RGD v2.0, any prior Appendix, or any UI/UX Screen Specification. Every module, entity, Functional Requirement, Business Rule, and screen referenced below is reused by name/ID from those documents, which remain authoritative for business and logical design.

1.1 Relationship to Prior Documents

Appendix-I through Appendix-N established the system's logical architecture, internal design, API shape, deployment topology, test strategy, and infrastructure — all technology-neutral. The eleven UI/UX Screen Specifications (Admission through Reports & Analytics) established every screen's structure and behavior, also technology-neutral. This MDB is the first document in the set that commits to specific technology choices, and it does so without altering any business rule, entity, or screen behavior already specified — it only fixes how those specifications are physically realized in code.

1.2 Scope of Part-1

Part-1 covers project foundation: technology stack, architecture, folder/package structure, module dependency, naming conventions, and coding standards. It does not cover entity/table design (Appendix-H remains authoritative for the logical model; physical schema mapping is a later Part), API endpoint design (Appendix-K remains authoritative for logical API groups; concrete endpoint contracts are a later Part), or CI/CD pipeline configuration (Appendix-L remains authoritative for logical deployment; concrete pipeline definition is a later Part).

2. Project Objectives

- Deliver the 11 approved modules (Admission, Student Information, Attendance, Examination, Fees, Library, Transport, HR & Payroll, Communication, Reports, Administration) exactly as specified in Appendix-E (Functional Requirements) and the corresponding UI/UX Screen Specifications, with zero unapproved scope additions.
- Enforce every one of the 71 Business Rules (Appendix-C v1.1) at the correct architectural layer, consistent with the Service-layer enforcement principle already established in Appendix-J Section 3.
- Meet every applicable Non-Functional Requirement target (Appendix-F) — performance, scalability, security, availability — using the technology stack fixed in Section 3.
- Provide a single, unambiguous implementation reference so that any developer or AI coding assistant joining the project produces code consistent with every other contributor, without needing to re-derive architectural or stylistic decisions independently.

- Maintain full, unbroken traceability from RGD v2.0 through to running code — every package, module, and component in the codebase should be traceable back to a specific FR, BR, NFR, entity, or screen ID.

3. Technology Stack

The following stack is fixed for the entire project. No component below is substitutable without a formal revision to this MDB.

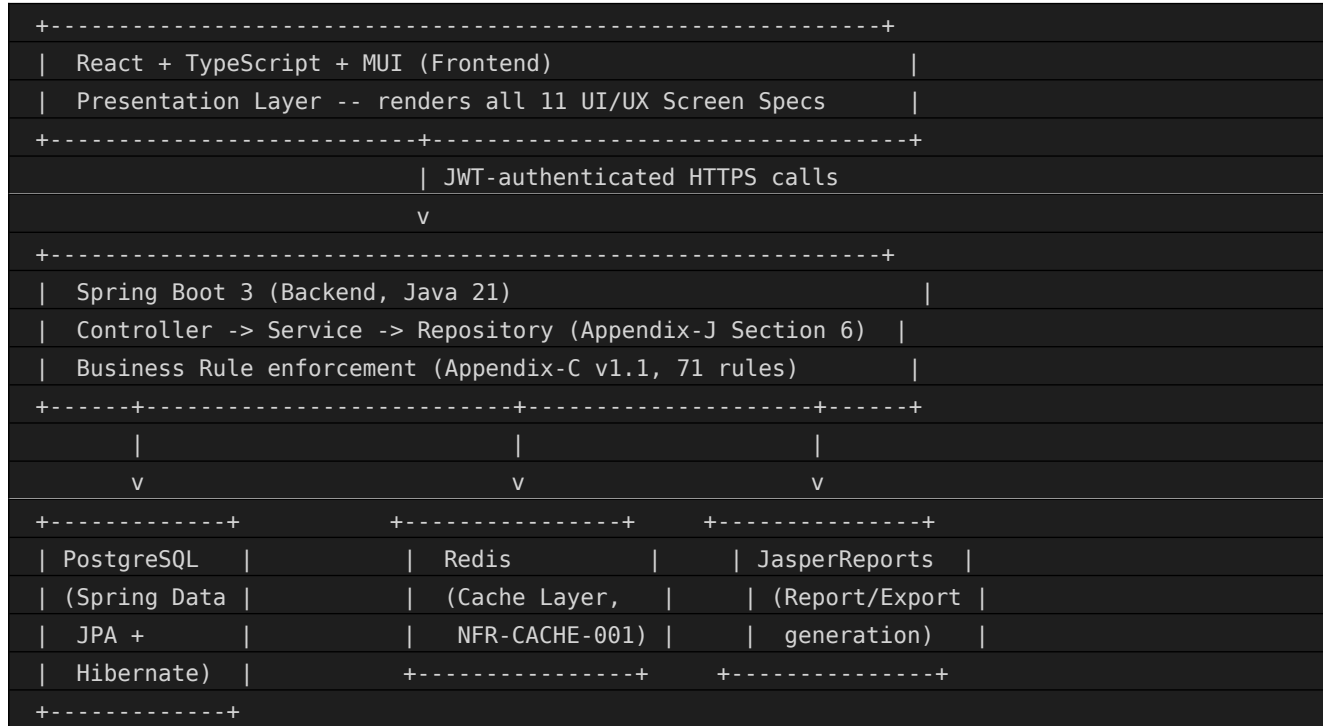
Layer	Technology	Role
Frontend	React + TypeScript + Material UI (MUI)	Presentation layer (Appendix-I Section 7) — renders all screens specified in the 11 UI/UX Screen Specifications
Backend	Java 21 + Spring Boot 3	Application/Business Logic layer (Appendix-I Section 7) — hosts the Controller-Service-Repository components (Appendix-J Section 6)
Database	PostgreSQL	Database/Persistence layer (Appendix-I Section 7) — physical realization of the 39 entities defined in Appendix-H
ORM	Spring Data JPA + Hibernate	Data Access layer mapping (Appendix-J Section 5, Repository component) between Java entities and PostgreSQL tables
Authentication	JWT + Refresh Token	Concrete realization of the Authentication Flow (Appendix-K Section 4) and token-based identity model (Appendix-I Section 9)
Cache	Redis	Concrete realization of the Cache Layer (Appendix-I Section 17, Appendix-L Section 6.7, NFR-CACHE-001)
Reports	JasperReports	Concrete realization of the Reports & Analytics module's export/generation capability (Appendix-K Section 18, Reports API Group)
Build	Maven	Backend build and dependency management
Version Control	Git	Source control for the entire monorepo (Section 5)

3.1 Stack-to-Architecture Mapping

Every technology choice above maps onto a tier or component already defined in Appendix-I's Layered Architecture and Appendix-L's Deployment Architecture — this MDB does not introduce any new architectural tier, only names the concrete technology occupying each already-defined tier.

4. High Level Architecture

Directly maps Appendix-I Section 7 (Layered Architecture) and Appendix-L Section 5 (Deployment Topology) onto the fixed technology stack.



This diagram is a technology-specific instantiation of Appendix-I's logical deployment tiers and Appendix-L's infrastructure components — it introduces no new tier, only names the concrete product occupying each tier already defined in those documents.

5. Project Folder Structure

Top-level repository layout. A single monorepo containing both frontend and backend is assumed for Part-1; this may be revisited in a later Part if the client/engineering team prefers separate repositories.

school-erp/	
-- backend/	(Java 21 + Spring Boot 3, Maven project -- Section 6)
-- frontend/	(React + TypeScript + MUI project -- Section 7)
-- docs/	
-- appendices/	(RGD v2.0, Appendix-A to N -- reference copies only, not edited here)
-- uiux-specs/	(11 approved UI/UX Screen Specifications -- reference copies only)
-- mdb/	(this Master Development Blueprint and future Parts)
-- scripts/	(build/setup helper scripts, non-CI/CD)
-- .gitignore	
-- README.md	
`-- LICENSE	

The docs/ directory holds reference copies of already-approved documents for developer convenience only — it is never the source of truth for those documents, and nothing in docs/ is regenerated or edited as part of implementation work.

6. Backend Package Structure

Package-by-module, then by architectural layer within each module — directly mirrors the module list (Appendix-E, Appendix-J Section 4) and the Controller-Service-Repository pattern (Appendix-J Section 6).

com.schoolerp	
-- admission/	(Appendix-J Section 4.1 -- FR-01 to FR-05c)
-- controller/	
-- service/	
-- repository/	
-- dto/	
-- entity/	
-- studentinformation/	(Appendix-J Section 4.2 -- FR-06 to FR-09)
-- attendance/	(Appendix-J Section 4.3 -- FR-10 to FR-13)
-- examination/	(Appendix-J Section 4.4 -- FR-17 to FR-21)
-- fee/	(Appendix-J Section 4.5 -- FR-22 to FR-26b)
-- library/	(Appendix-J Section 4.6 -- FR-27 to FR-29)
-- transport/	(Appendix-J Section 4.7 -- FR-30 to FR-32)
-- hrpayroll/	(Appendix-J Section 4.8 -- FR-33 to FR-36)
-- communication/	(Appendix-J Section 4.9 -- FR-37 to FR-39)
-- reports/	(Appendix-J Section 4.10 -- FR-40 to FR-42)
-- administration/	(Appendix-J Section 4.11 -- shared services:
	AuditLogService, ConfigurationService,
	ApprovalRequestService, DocumentService,
	NotificationService, UserService, RoleService)
-- common/	
-- config/	(Spring configuration, incl. Redis, JPA)
-- security/	(JWT/Refresh Token handling -- Appendix-K Section 4)
-- exception/	(Error Response Standard -- Appendix-K Section 8)
-- util/	

Every module package follows the identical five-folder internal shape (controller/service/repository/dto/entity), consistent with the Component Breakdown already defined in Appendix-J Section 5 — this uniformity is mandatory, not a per-module choice.

7. Frontend Folder Structure

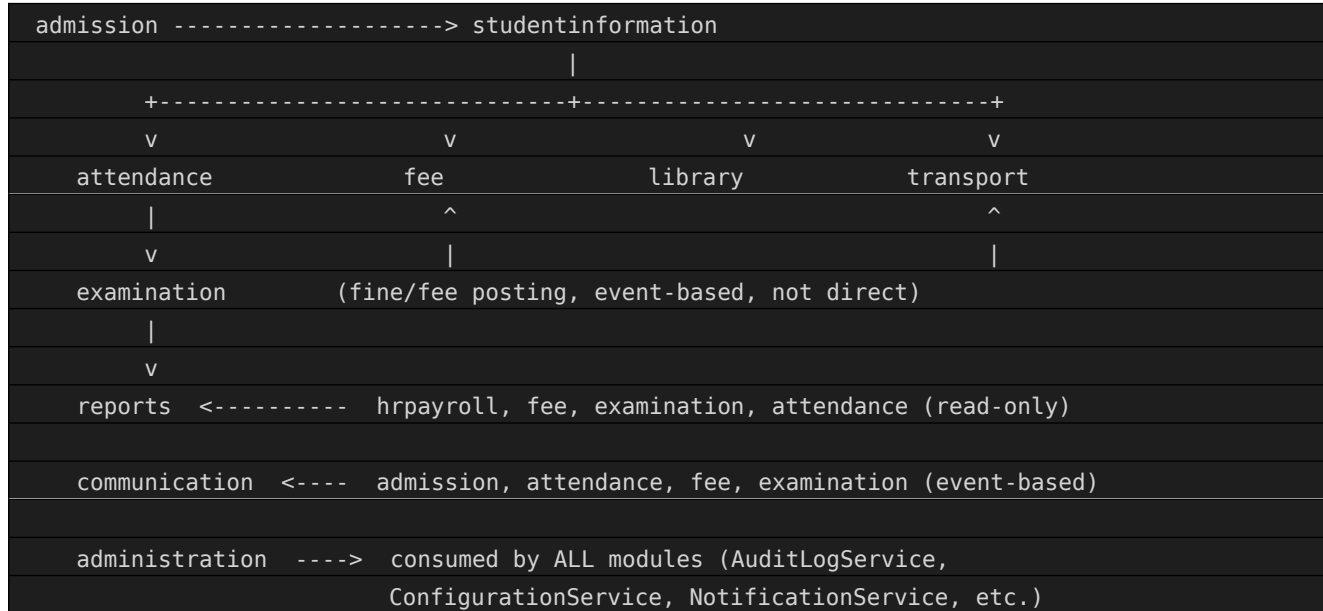
Module-by-feature layout under `src/modules/`, one folder per module, directly mirroring the 11 UI/UX Screen Specifications.

<code>frontend/src/</code>	
<code> -- modules/</code>	
<code> </code>	<code> -- admission/ (SCR-ADM-001 to 005)</code>
<code> </code>	<code> -- student-information/ (SCR-SIS-001 to 013)</code>
<code> </code>	<code> -- attendance/ (SCR-ATT-001 to 012)</code>
<code> </code>	<code> -- examination/ (SCR-EXM-001 to 014)</code>
<code> </code>	<code> -- fee/ (SCR-FEE-001 to 014)</code>
<code> </code>	<code> -- library/ (SCR-LIB-001 to 014)</code>
<code> </code>	<code> -- transport/ (SCR-TRN-001 to 012)</code>
<code> </code>	<code> -- hr-payroll/ (SCR-HR-001 to 012)</code>
<code> </code>	<code> -- communication/ (SCR-COM-001 to 010)</code>
<code> </code>	<code> -- reports/ (SCR-RPT-001 to 010)</code>
<code> </code>	<code>`-- administration/ (shared: user/role/config screens)</code>
<code> -- components/ (shared MUI-based UI components, reused across modules)</code>	
<code> -- services/ (API client layer, one file per backend module)</code>	
<code> -- hooks/ (shared React hooks)</code>	
<code> -- store/ (application state management)</code>	
<code> -- routes/ (route definitions, mapped to Navigation Path per screen)</code>	
<code> -- theme/ (MUI theme configuration)</code>	
<code> -- types/ (shared TypeScript types/interfaces)</code>	
<code> -- utils/</code>	
<code> -- App.tsx</code>	
<code>`-- main.tsx</code>	

Each module folder under `src/modules/` contains one component file per screen ID already catalogued in its corresponding UI/UX Screen Specification — the folder structure is a direct, traceable mirror of that specification, not an independent design.

8. Module Dependency

Directly reuses the module dependency chain already established in Appendix-D and Appendix-I Section 6, expressed here as backend package dependency direction.



- A module's service package may depend on another module's published service interface, but never on another module's repository or entity package directly — this enforces the same module-boundary discipline already established in Appendix-J Section 3 and Appendix-I Section 7.4.
- Cross-module effects (Library fine to Fee, Transport fee linkage) are implemented as Spring application events or an equivalent publish/subscribe mechanism, not direct cross-package method calls — consistent with the Event Publisher/Consumer component already defined in Appendix-J Section 5.
- The administration package is the only package every other module package is permitted to depend on directly, consistent with its shared-service role already established in Appendix-J Section 4.11.

9. Naming Conventions

Extends the database naming conventions already fixed in Appendix-H Section 2.2 and Appendix-N Section 2.12 with the Java/TypeScript-specific conventions needed for this stack.

Element	Convention	Example
Java Class (Controller/Service/etc.)	PascalCase, suffixed by role	AdmissionController, PaymentService, InvoiceRepository
Java Method	camelCase, verb-first	createApplication(), calculateLateFee()
Java Package	all-lowercase, module name	com.schoolerp.fee
REST Endpoint Path	kebab-case, resource-oriented (Appendix-K Section 3)	/admissions, /fee-structures
Database Table/Column	snake_case (Appendix-H Section 2.2, unchanged)	students, admission_number

Element	Convention	Example
React Component File	PascalCase.tsx, matches component name	StudentList.tsx, FeeReceipt.tsx
React Hook	camelCase, "use" prefix	useStudentSearch(), useAuth()
TypeScript Interface/Type	PascalCase, descriptive suffix	StudentDto, InvoiceResponse
State Store Slice	camelCase, module-scoped	admissionSlice, feeSlice
Entity ID Cross-Reference	Unchanged from Appendix-G/H (e.g., ENT-SIS-001)	Retained in code comments/doc-strings for traceability

10. Coding Standards

10.1 Backend (Java 21 + Spring Boot 3)

- Constructor-based dependency injection only — field injection is not permitted, for testability consistent with Appendix-M's Unit Testing level.
- Every Controller method delegates to exactly one Service method and contains no business logic itself, directly enforcing the Controller responsibility already defined in Appendix-J Section 5.
- Every Service method that changes state calls AuditLogService before returning, per Appendix-J Section 10's mandatory (not optional) logging requirement.
- DTOs are used at every Controller boundary — JPA entities are never returned directly from a Controller, preserving the Appendix-K Section 6/7 Common Request/Response Structure independent of the persistence model.
- Exceptions follow the categories already defined in Appendix-J Section 20 (Validation, Business Rule, Integration, Concurrency, Unhandled) — each category maps to one custom exception type and one centralized handler.
- Unit tests (JUnit 5 + Mockito) are required for every Service method implementing a Business Rule (Appendix-C v1.1), consistent with Appendix-M Section 8's Unit Testing level and Section 5's Shift-Left principle.

10.2 Frontend (React + TypeScript + MUI)

- Functional components with Hooks only — class components are not permitted.
- TypeScript strict mode is enabled project-wide; the `any` type is not permitted except at a documented, reviewed integration boundary.
- Every screen component is named and organized to match its Screen ID from the corresponding UI/UX Screen Specification (e.g., SCR-FEE-005 Fee Collection maps to a FeeCollection.tsx component), preserving direct traceability.
- Shared, cross-module UI elements (buttons, tables, dialogs) are built once under components/ using MUI primitives, not re-implemented per module.
- API calls are isolated to the services/ layer — no component calls a backend endpoint directly, mirroring the Controller boundary discipline on the backend.

10.3 Cross-Cutting

- Linting (ESLint + Prettier for frontend; Checkstyle or equivalent for backend) is enforced in the build; a build with lint errors does not pass Appendix-M's Entry Criteria for Integration Testing.
- Every commit message references the Screen ID, FR ID, or BR ID it implements, preserving the traceability chain from RGD v2.0 through to the Git history itself.
- No code is merged without passing the Unit Testing level (Appendix-M Section 8) for the changed module, consistent with the project's Shift-Left testing principle (Appendix-M Section 5).